

Run Your Own Local Language Model

A hands-on guide to getting a local LLM running on your machine in under 30 minutes — and, if you want to go further, the vibe guidelines for building your own agent harness from scratch.

PREREQUISITE

macOS or Linux, 8 GB RAM

PART 1

~20 min

PART 2

1–3 days

COST

Free

PART 1 *Getting Ollama running*

~20 min

Ollama is an open-source tool that makes running local language models as easy as running a Docker container. It handles downloading the model weights, serving them via a local API, and providing a simple CLI. You do not need a GPU — it runs on CPU, though it is faster on Apple Silicon.

1 Download and install Ollama

Go to **ollama.com** and download the macOS or Linux installer. On Mac, drag it to Applications and open it — you will see an icon in the menu bar. On Linux, run the install script.

LINUX INSTALL (ONE LINE)

```
$ curl -fsSL https://ollama.com/install.sh | sh
```

NOTE

After installing on Mac, Ollama runs as a background service automatically. You will see a llama icon in your menu bar.

2 Pull the Llama 3.2 model

Open Terminal and pull a 3 billion parameter Llama model. This downloads about 2 GB — the model weights are stored locally at `~/.ollama/models`. You only need to do this once.

TERMINAL

```
$ ollama pull llama3.2
pulling manifest...
pulling 966de95ca8a6... ██████████ 100%
success
```

MODEL SIZES

llama3.2 is 3B parameters (~2 GB). If you have 16 GB RAM and want better quality, try `ollama pull llama3.2:latest` or `ollama pull llama3.1:8b` (~4.7 GB).

3 Run your first prompt

Run the model interactively from the terminal. Type a prompt and press Enter. The model responds entirely locally — no internet connection required after the initial download.

```
TERMINAL — INTERACTIVE MODE
```

```
$ ollama run llama3.2
```

```
>>> What is a large language model, in two sentences?
```

To exit interactive mode, type `/bye` or press `Ctrl+D`. To run a one-shot prompt without entering interactive mode:

```
ONE-SHOT PROMPT
```

```
$ ollama run llama3.2 "Summarise the key ideas behind RAG in plain English"
```

4 Try it from code

Ollama also exposes a local REST API on port 11434. You can call it from any language. Here is a minimal JavaScript example to try in Node.js:

```
NODE.JS
```

```
const res = await fetch('http://localhost:11434/api/generate', {  method: 'POST',  
  body: JSON.stringify({  model: 'llama3.2',  prompt: 'What does an AI agent actually  
do?',  stream: false  }) }); const { response } = await res.json();  
console.log(response);
```

```
WHAT THIS TEACHES YOU
```

The model is just an HTTP endpoint. Any application — a Node server, a Python script, a shell script — can call it. This is the foundation of everything that follows.

Prompts worth trying

These are designed to help you develop an intuition for what local models are good at and where they fall short.

```
REASONING
```

```
"I have a dataset of A/B test results. Walk me through how you would decide whether a 2% lift is  
statistically significant."
```

```
SUMMARISATION
```

```
"Summarise the following meeting notes in three bullet points: [paste text]"
```

```
CODE GENERATION
```

```
"Write a Node.js function that takes an array of numbers and returns the mean, median, and  
standard deviation."
```

LIMITS TEST

```
"What is today's date?"
```

WHAT TO NOTICE

The model is strong at reasoning, summarisation, and code. It has no knowledge of current events and no access to the internet. It cannot call your calendar or read your files — unless you give it tools.

PART 2 *Building your own harness*

1-3 days

A harness is the layer between the model and the real world. It handles tool calls, manages context, enforces governance, and connects to real data sources. You do not need to build everything at once — start with the core loop and add pieces as you understand them.

What you are building

Server

Receives tasks, manages the agent loop, talks to Ollama, executes tools, stores state.

```
Node.js + Fastify · SQLite (better-sqlite3)
```

Agent loop

Sends prompt to model, receives tool call or final answer, executes tool, feeds result back. Repeats until done.

```
Ollama /api/chat with tools param
```

Tools / connectors

Functions the model can call — read files, search knowledge, call APIs. Each has a JSON schema so the model knows how to invoke it.

```
Zod for validation · keytar for secrets
```

Gate system

Intercepts sensitive tool calls before execution. Stores them as pending — a human approves or denies before the loop continues.

```
DB table + SSE for real-time push
```

Knowledge base

Indexes local files as vector embeddings. Agent runs a semantic search at query time and receives the most relevant chunks as context.

```
nomic-embed-text · sqlite-vec
```

Client UI

Browser interface — chat console, agent config, connection management, gate queue, workflow viewer.

```
React + Vite · no component library
```

How to build it — vibe coding guidelines

Open Claude Code (or any AI coding assistant) in a new empty directory and work through these prompts in order. Each one builds on the last. Read the output before moving on — understanding what was built is as important as building it.

STEP 1 · CORE SERVER AND AGENT LOOP

PROMPT TO CLAUDE CODE

"Create a Node.js server using Fastify that exposes a POST /api/runs endpoint. When called with a task string, it should start an agent loop: send the task to a local Ollama model (llama3.2) using the /api/chat endpoint with tool_call support, execute any tool calls the model makes, feed results back into the conversation, and repeat until the model produces a final text response. Log each step. Store runs in a SQLite database using better-sqlite3. The only tool for now should be echo(message) which just returns the message."

STEP 2 · FILE SYSTEM TOOLS

PROMPT TO CLAUDE CODE

"Add two tools to the agent loop: fs.readFile(path) and fs.listDirectory(path). Both should be constrained to a set of allowed scopes (directory paths) — if the requested path is not inside an allowed scope, return an error. Add a scopes field to the agent configuration in the database. Write Zod schemas for the tool parameters."

STEP 3 · GATE SYSTEM

PROMPT TO CLAUDE CODE

*"Before any tool call executes, check it against a rules table in the database. Each rule has a pattern (glob-style, e.g. 'fs.write **'), a policy (allow / gate / block), and a timeout. If the policy is 'gate', pause the agent loop, save the pending action to a gates table, and wait for an HTTP POST to /api/gates/:id/approve or /api/gates/:id/deny before continuing. Use Server-Sent Events to push gate events to the browser in real time."*

STEP 4 · KNOWLEDGE BASE

PROMPT TO CLAUDE CODE

"Add a knowledge base system. Use Ollama to generate embeddings with the nomic-embed-text model. Index markdown and text files from configured folder paths — split them into chunks, embed each chunk, and store in SQLite using the sqlite-vec extension for vector similarity search. Add a kb.query(question) tool the agent can call to retrieve the top-k most relevant chunks. Add a /api/knowledge endpoint to manage folders and trigger re-indexing."

STEP 5 · FIRST CONNECTOR (YOUR CHOICE)

PROMPT TO CLAUDE CODE

"Add a Gmail connector using OAuth 2.0 authorization code flow. The redirect URI should be http://localhost:8474/oauth/gmail/callback. Store tokens in the OS keychain using keytar. Expose three tools: gmail.listEmails, gmail.readEmail, and gmail.searchEmails. Each tool should handle token refresh automatically. Add API routes to start the OAuth flow, handle the callback, check connection status, and disconnect."

STEP 6 · BROWSER UI

PROMPT TO CLAUDE CODE

"Create a React + Vite frontend for the harness. It needs: a Console screen to send tasks and watch agent runs in real time (tool calls, responses, gate events via SSE), an Agents screen to configure agent permissions and knowledge scopes, a Connections screen to manage integrations, and a Gates screen to approve or deny pending gate actions. Use no component library — write all styles inline. The visual style should be minimal and dark."

Before you run each prompt: Read the previous output first. Understand what was built, what files were created, and what assumptions were made. AI coding assistants write coherent code but they make architectural choices you will need to live with. Redirect early rather than refactor late.

Things to know before you start

ON MODEL CHOICE

Start with llama3.2 (3B). It is fast, fits in 8 GB RAM, and handles tool calls reliably enough for a demo. Upgrade to llama3.1:8b or llama3.3:70b when you want better reasoning quality and have the hardware for it.

ON TOOL CALL RELIABILITY

Small models (3B) occasionally pass wrong parameter types — a string where a boolean is expected, for instance. Write Zod schemas with coercion: `z.union([z.boolean(), z.string().transform(v => v === 'true')])`. Larger models are more reliable but the defensive validation is good practice regardless.

ON THE GATE SYSTEM

Build the gate system before you build any write tools. The temptation is to get things working first and add governance later. That approach makes the governance feel like a constraint on a working system, when the correct mental model is the opposite: the gate is what makes the write tools safe to include at all.

ON CONTEXT WINDOW MANAGEMENT

The agent loop appends every tool result to the conversation. Long runs on 3B models will eventually exceed the context window and produce degraded responses. Add a `max_tool_calls` limit to each run and a token counter. Stop gracefully rather than silently degrading.

ON SECRETS

Use the OS keychain (keytar) for tokens and API keys — never the database. The database may be committed or backed up. The keychain will not be.

